I

nsorFlow Roadmap: Beginner to Very Advanced

Set 0 — Setup and Environment

Goal: Prepare your TensorFlow environment.

1. Install TensorFlow.
2. Verify TensorFlow version.
3. Check CPU/GPU availability.
4. Learn how to create a clean virtual environment.
5. Learn basic TensorFlow import pattern:

import tensorflow as tf

6. Run your first TensorFlow operation.
7. Understand tf.Tensor.
8. Understand TensorFlow eager execution.
9. Learn difference between CPU execution and GPU execution.
10. Learn how to check available devices.

Official TensorFlow installation supports pip-based installation and has separate guidance for stable, nightly, and CPU-only packages.

Set 1 — TensorFlow Core Basics

Goal: Understand TensorFlow objects before using models.

Learn these one by one:

1. tf.constant
2. tf.Variable
3. Tensor shape
4. Tensor rank

5. Tensor dtype
6. Tensor indexing
7. Tensor slicing
8. Tensor broadcasting
9. Tensor reshaping
10. Tensor casting
11. Tensor math operations
12. Tensor comparison operations
13. Tensor reduction operations:

- tf.reduce_sum
- tf.reduce_mean
- tf.reduce_max
- tf.reduce_min

14. Tensor random values:

- tf.random.normal
- tf.random.uniform

15. Difference between immutable tensor and mutable variable.

Practice task:
Create weight and bias manually using tf.Variable, then calculate:

y_pred = weight * x + bias

Set 2 — Automatic Differentiation

Goal: Understand how TensorFlow calculates gradients.

Learn:

1. What is automatic differentiation?
2. What is tf.GradientTape?
3. Why TensorFlow needs gradients.
4. How gradient descent works inside TensorFlow.
5. How to calculate derivative of a simple equation.
6. How to calculate gradient of loss with respect to weight and bias.
7. How to apply gradients manually.
8. How backpropagation uses gradients.

TensorFlow's official autodiff guide explains that automatic differentiation is useful for implementing machine learning algorithms such as backpropagation.

Practice task:
Manually train a simple linear model:

y = wx + b

Use:

```
with tf.GradientTape() as tape:
    ...
```

Set 3 — Manual Linear Regression in TensorFlow

Goal: Connect your existing regression knowledge with TensorFlow.

Learn:

1. Define weight manually.
2. Define bias manually.
3. Predict using TensorFlow operation.
4. Define MSE loss.
5. Calculate gradient using GradientTape.
6. Update weight manually.
7. Update bias manually.
8. Run training loop.
9. Print loss per epoch.
10. Predict unseen data.
11. Compare actual vs predicted.
12. Understand what TensorFlow is doing internally.

Practice task:
Build linear regression without Keras.

You should be able to explain:

loss → gradient → update → repeat

Set 4 — TensorFlow Optimizers

Goal: Learn how TensorFlow updates parameters.

Learn:

1. tf.keras.optimizers.SGD
2. Learning rate
3. Large learning rate problem
4. Small learning rate problem
5. Adam optimizer
6. RMSprop optimizer
7. Optimizer state
8. optimizer.apply_gradients
9. Difference between manual update and optimizer update
10. When to use SGD
11. When to use Adam

Practice task:
Train the same linear model with:

tf.keras.optimizers.SGD()

Then train it again with:

tf.keras.optimizers.Adam()

Compare convergence behavior.

Set 5 — Keras Sequential API

Goal: Learn the easiest way to build models in TensorFlow.

Learn:

1. What is Keras?
2. What is tf.keras.Sequential?
3. What is a layer?

4. What is Dense layer?
5. What is input shape?
6. What is output unit?
7. What is activation?
8. What is model.compile()?
9. What is model.fit()?
10. What is model.evaluate()?
11. What is model.predict()?
12. What is loss?
13. What is optimizer?
14. What is metric?

TensorFlow describes Keras as the high-level API of the TensorFlow platform, designed for productive ML workflows and fast experimentation.

Practice task:
Build these models:

1. Linear regression using Sequential
2. Binary classification using Sequential
3. Multiclass classification using Sequential

Set 6 — Keras Functional API

Goal: Move beyond simple sequential models.

Learn:

1. Why Sequential API is limited.
2. What is Functional API?
3. tf.keras.Input
4. Connecting layers manually
5. Multiple input models
6. Multiple output models
7. Shared layers
8. Branching architecture
9. Model summary
10. Difference between Sequential and Functional API

Practice task:
Build a model like this:

Input → Dense → Dense → Output

Then build:

Input → Branch A
→ Branch B
→ Concatenate → Output

Set 7 — Model Subclassing

Goal: Learn advanced model design.

Learn:

1. What is subclassing?
2. class MyModel(tf.keras.Model)
3. __init__()
4. call()
5. Define layers inside class.
6. Forward pass inside call.
7. Difference between Functional API and subclassing.
8. When subclassing is useful.
9. Debugging subclassed models.
10. Saving limitations and best practices.

Practice task:
Create your own class-based neural network.

```
class MyModel(tf.keras.Model):
    def __init__(self):
        super().__init__()
        self.dense1 = tf.keras.layers.Dense(32, activation="relu")
        self.out = tf.keras.layers.Dense(1)

    def call(self, x):
```

```
        x = self.dense1(x)
        return self.out(x)
```

Set 8 — Layers Deep Dive

Goal: Understand common TensorFlow layers.

Learn:

1. Dense
2. Dropout
3. BatchNormalization
4. Flatten
5. Reshape
6. Embedding
7. Conv2D
8. MaxPooling2D
9. GlobalAveragePooling2D
10. LSTM
11. GRU
12. Bidirectional
13. LayerNormalization
14. Custom layer creation

Practice task:
Create a custom layer:

```
class CustomDense(tf.keras.layers.Layer):
    ...
```

Set 9 — Loss Functions

Goal: Learn which loss to use for which problem.

Learn:

1. Mean Squared Error

2. Mean Absolute Error
3. Binary Crossentropy
4. Categorical Crossentropy
5. Sparse Categorical Crossentropy
6. Huber Loss
7. Custom loss function
8. Loss reduction
9. Loss curve interpretation
10. Difference between loss and metric

Practice task:
Train one classification model using:

BinaryCrossentropy

Then train multiclass model using:

SparseCategoricalCrossentropy

Set 10 — Metrics

Goal: Evaluate models properly.

Learn:

1. Accuracy
2. Binary accuracy
3. Categorical accuracy
4. Precision
5. Recall
6. AUC
7. Mean absolute error
8. Mean squared error
9. Custom metric
10. Training metric vs validation metric

Practice task:
Use multiple metrics in model.compile():

metrics=["accuracy", tf.keras.metrics.Precision(), tf.keras.metrics.Recall()]

Set 11 — Activation Functions

Goal: Understand non-linearity.

Learn:

1. Linear activation
2. Sigmoid
3. Tanh
4. ReLU
5. LeakyReLU
6. ELU
7. Softmax
8. When to use sigmoid
9. When to use softmax
10. Why hidden layers usually use ReLU
11. Vanishing gradient problem
12. Exploding gradient problem

Practice task:
Train the same model with:

relu
tanh
sigmoid

Compare training behavior.

Set 12 — Training Control

Goal: Learn how to control training.

Learn:

1. Epoch

2. Batch size
3. Steps per epoch
4. Validation split
5. Validation data
6. Shuffling
7. Overfitting
8. Underfitting
9. Early stopping
10. Model checkpointing
11. Learning rate scheduler
12. ReduceLROnPlateau
13. Callback system

Practice task:
Use callbacks:

tf.keras.callbacks.EarlyStopping
tf.keras.callbacks.ModelCheckpoint
tf.keras.callbacks.ReduceLROnPlateau

Set 13 — TensorBoard

Goal: Track model training professionally.

Learn:

1. What is TensorBoard?
2. How to log loss.
3. How to log accuracy.
4. How to compare runs.
5. How to view model graph.
6. How to inspect histograms.
7. How to track embeddings.
8. How to organize experiment folders.

TensorBoard is TensorFlow's tool for tracking metrics such as loss and accuracy, visualizing graphs, and supporting the ML workflow.

Practice task:
Train 3 models with different learning rates and compare them in TensorBoard.

Set 14 — tf.data Pipeline

Goal: Learn production-style data loading.

Learn:

1. What is tf.data.Dataset?
2. Create dataset from tensors.
3. batch()
4. shuffle()
5. map()
6. prefetch()
7. cache()
8. repeat()
9. Train model using dataset object.
10. Build efficient input pipeline.
11. Load image folders.
12. Load text data.
13. Load CSV using TensorFlow tools.
14. Performance optimization.

TensorFlow tutorials use tf.data for loading different data formats and building input pipelines.

Practice task:
Convert your raw training data into:

tf.data.Dataset

Then train using:

model.fit(dataset)

Set 15 — Image Classification

Goal: Enter computer vision with TensorFlow.

Learn:

1. Image tensor shape
2. RGB channels
3. Image resizing
4. Image normalization
5. Image augmentation
6. CNN basics
7. Conv2D
8. MaxPooling2D
9. Flatten
10. GlobalAveragePooling2D
11. Binary image classification
12. Multiclass image classification
13. Overfitting in CNN
14. Dropout in CNN
15. Batch normalization in CNN

Practice projects:

1. Cat vs dog classifier
2. Medical image normal/abnormal classifier
3. Fruit classifier
4. Handwritten digit classifier

Set 16 — Transfer Learning

Goal: Use pretrained models.

Learn:

1. What is transfer learning?
2. Why pretrained models help.
3. Feature extraction

4. Fine-tuning
5. Freezing layers
6. Unfreezing layers
7. Lower learning rate during fine-tuning
8. Using MobileNet
9. Using EfficientNet
10. Using ResNet
11. Image preprocessing for pretrained models
12. Avoiding overfitting with small data

TensorFlow's guide explains transfer learning as using features learned from one problem to help solve a new similar problem, especially when the new dataset is too small to train from scratch.

Practice task:
Use a pretrained CNN and train only the final classification layer first. Then unfreeze some upper layers and fine-tune.

Set 17 — Text Classification

Goal: Learn NLP basics in TensorFlow.

Learn:

1. Text vectorization
2. Tokenization
3. Vocabulary
4. Sequence length
5. Padding
6. TextVectorization layer
7. Embedding layer
8. Simple dense model for text
9. LSTM for text
10. GRU for text
11. Bidirectional LSTM
12. Text classification
13. Sentiment analysis

14. Multiclass text classification

Practice projects:

1. Spam detection
2. Sentiment analysis
3. News category classification
4. Customer complaint classifier

Set 18 — Time Series

Goal: Learn sequential prediction.

Learn:

1. What is time series?
2. Windowing
3. Forecast horizon
4. Univariate forecasting
5. Multivariate forecasting
6. Dense model for time series
7. CNN for time series
8. LSTM for time series
9. GRU for time series
10. Sequence-to-one prediction
11. Sequence-to-sequence prediction
12. Walk-forward validation
13. Forecast error analysis

Practice projects:

1. Temperature prediction
2. Sales forecasting
3. Stock-like synthetic sequence forecasting
4. Sensor data prediction

Set 19 — Custom Training Loop

Goal: Go beyond model.fit().

Learn:

1. Why custom loop is needed.
2. Manual epoch loop.
3. Manual batch loop.
4. Forward pass.
5. Loss calculation.
6. Gradient calculation.
7. Optimizer update.
8. Metric update.
9. Validation loop.
10. Saving best model manually.
11. Logging manually.
12. Debugging gradients.

TensorFlow's basic training loop guide describes the training loop as: send batch through model, calculate loss, use gradient tape to find gradients, and optimize variables.

Practice task:
Train a neural network without model.fit().

Set 20 — tf.function and Graph Mode

Goal: Learn performance-oriented TensorFlow.

Learn:

1. Eager execution
2. Graph execution
3. What is tf.function
4. AutoGraph
5. Tracing
6. Retracing problem

7. Input signatures
8. Python side effects inside tf.function
9. Debugging graph mode
10. Performance improvement

Practice task:
Convert your custom training step into:

```
@tf.function
def train_step(...):
    ...
```

TensorFlow 2 focuses on ease of use with eager execution and high-level APIs, while also supporting flexible model building and graph execution patterns.

Set 21 — Regularization

Goal: Reduce overfitting.

Learn:

1. L1 regularization
2. L2 regularization
3. Dropout
4. Early stopping
5. Data augmentation
6. Batch normalization
7. Label smoothing
8. Weight decay
9. Small model vs large model
10. Train/validation gap analysis

Practice task:
Train an overfitting model first. Then reduce overfitting using:

Dropout
L2

EarlyStopping
Data augmentation

Set 22 — Hyperparameter Tuning

Goal: Improve model performance systematically.

Learn:

1. Learning rate tuning
2. Batch size tuning
3. Number of layers
4. Number of neurons
5. Activation choice
6. Optimizer choice
7. Dropout rate
8. Regularization strength
9. Search space
10. Manual tuning
11. KerasTuner basics
12. Experiment tracking with TensorBoard

Practice task:
Tune one model using 5 different learning rates and 3 different batch sizes.

Set 23 — Saving and Loading Models

Goal: Make models reusable.

Learn:

1. Save full model.
2. Save only weights.
3. Load full model.
4. Load weights.
5. Checkpoint during training.

6. Resume training.
7. Export for inference.
8. SavedModel format.
9. Keras model format.
10. Versioning models.

TensorFlow's save/load guide explains that saving allows training to resume later and allows others to recreate the model; SavedModel stores serialized signatures and state such as variables and vocabularies.

Practice task:
Train a model, save it, close the notebook/script, reload it, and predict unseen data.

Set 24 — Model Inference

Goal: Use trained models in real applications.

Learn:

1. Single input prediction
2. Batch prediction
3. Preprocessing before prediction
4. Postprocessing output
5. Probability interpretation
6. Threshold selection
7. Class label mapping
8. Inference speed
9. Model warm-up
10. Input validation

Practice task:
Create a function:

```
def predict_user_input(input_data):
    ...
```

Set 25 — Deployment with TensorFlow Serving

Goal: Serve TensorFlow models in production.

Learn:

1. What is TensorFlow Serving?
2. Export model.
3. SavedModel directory structure.
4. Serve model using Docker.
5. REST API request.
6. gRPC request.
7. Model versioning.
8. Updating model without changing app code.
9. Calling model from backend service.
10. Production inference pipeline.

TensorFlow Serving is described by TensorFlow as a flexible, high-performance serving system designed for production environments.

For you as a Spring Boot developer:
This part is very important. You can train with TensorFlow, export the model, serve it through TensorFlow Serving, then call it from your Spring Boot application using REST or gRPC.

Set 26 — TensorFlow Lite

Goal: Deploy models to mobile or edge devices.

Learn:

1. What is TensorFlow Lite?
2. Convert Keras model to TFLite.
3. TFLiteConverter
4. Quantization
5. Float16 quantization
6. Integer quantization
7. Model size reduction

8. Mobile inference
9. Edge device inference
10. Test converted model.

TensorFlow's tf.lite.TFLiteConverter is used to convert TensorFlow models into TensorFlow Lite models.

Practice task:
Train a small image classifier and convert it into .tflite.

Set 27 — Distributed Training

Goal: Train on multiple GPUs or machines.

Learn:

1. Why distributed training is needed.
2. tf.distribute.Strategy
3. MirroredStrategy
4. Multi-GPU training
5. Multi-worker training
6. TPU training concept
7. Synchronous training
8. Batch size adjustment
9. Distributed dataset
10. Saving distributed models

TensorFlow's tf.distribute.Strategy API supports training across multiple GPUs, multiple machines, or TPUs with relatively small changes to existing training code.

Practice task:
Use:

tf.distribute.MirroredStrategy()

Train the same model on available GPUs.

Set 28 — Mixed Precision Training

Goal: Improve speed and memory efficiency.

Learn:

1. Float32
2. Float16
3. Mixed precision
4. Loss scaling
5. GPU acceleration
6. Memory saving
7. When not to use mixed precision
8. Accuracy checking
9. Performance benchmarking

TensorFlow's mixed precision guide explains that using both 16-bit and 32-bit floating-point types can make training faster and use less memory while keeping numerically sensitive parts stable.

Practice task:
Train one CNN normally, then train it with mixed precision and compare training time.

Set 29 — Advanced Computer Vision

Goal: Move beyond simple image classification.

Learn:

1. Image classification
2. Multi-label classification
3. Object detection concept
4. Image segmentation concept
5. U-Net architecture
6. Autoencoders for images
7. Siamese networks
8. Face similarity model

9. Image embedding
10. Contrastive learning basics

Practice projects:

1. Skin lesion classification
2. X-ray normal/abnormal classifier
3. Road sign classifier
4. Image similarity search model

Set 30 — Advanced NLP with TensorFlow

Goal: Build stronger text models.

Learn:

1. Word embeddings
2. Sequence models
3. Attention concept
4. Transformer concept
5. Positional encoding
6. Text classification with transformer-like models
7. Sequence labeling
8. Named entity recognition
9. Text similarity
10. Fine-tuning pretrained text models

Practice projects:

1. Review sentiment model
2. Question classification model
3. Text similarity checker
4. Simple document classifier

Set 31 — Custom Layers, Custom Losses, Custom Metrics

Goal: Become independent in TensorFlow model design.

Learn:

1. Custom layer
2. Custom activation
3. Custom loss
4. Custom metric
5. Custom callback
6. Custom training step
7. Override train_step
8. Override test_step
9. Create reusable components
10. Package your model code cleanly

Practice task:
Create:

CustomDenseLayer
CustomRMSELoss
CustomF1Metric
CustomEarlyStopCallback

Set 32 — Debugging TensorFlow Models

Goal: Learn how to find mistakes.

Learn:

1. Shape mismatch
2. Wrong dtype
3. Wrong loss function
4. Wrong activation
5. NaN loss
6. Exploding gradient
7. Vanishing gradient
8. Overfitting
9. Underfitting
10. Bad learning rate
11. Dataset pipeline bugs

12. Inference preprocessing mismatch
13. Training-serving skew

Practice task:
Intentionally create these errors and fix them:

wrong input shape
wrong label shape
wrong loss
too high learning rate
missing normalization

Set 33 — Production ML Engineering with TensorFlow

Goal: Use TensorFlow in real systems.

Learn:

1. Project structure
2. Config files
3. Training script
4. Evaluation script
5. Inference script
6. Model registry concept
7. Model versioning
8. Logging
9. Reproducibility
10. Batch inference
11. Online inference
12. REST integration
13. Docker deployment
14. Monitoring prediction quality
15. Retraining pipeline concept

Spring Boot connection:
A good production design is:

Spring Boot App
↓ REST/gRPC
TensorFlow Serving
↓
SavedModel

Set 34 — Very Advanced TensorFlow Topics

Goal: Reach expert level.

Learn:

1. Advanced GradientTape
2. Higher-order gradients
3. Jacobian
4. Hessian concept
5. Custom optimizer
6. Custom training engine
7. Custom regularizer
8. Custom initializer
9. Graph optimization
10. XLA basics
11. Model pruning
12. Quantization-aware training
13. Knowledge distillation
14. Multi-task learning
15. Self-supervised learning
16. Contrastive learning
17. Generative models
18. Autoencoders
19. Variational autoencoders
20. GANs

TensorFlow has separate advanced guides for automatic differentiation beyond the basics, including deeper features of GradientTape.

Recommended Learning Order

Follow this exact order:

1. TensorFlow tensors and variables
2. GradientTape
3. Manual linear regression
4. Optimizers
5. Keras Sequential API
6. Functional API
7. Model subclassing
8. Layers, losses, metrics
9. Callbacks
10. TensorBoard
11. tf.data
12. Image classification
13. Text classification
14. Time series
15. Transfer learning
16. Custom training loops
17. tf.function
18. Saving/loading
19. TensorFlow Serving
20. TensorFlow Lite
21. Distributed training
22. Mixed precision
23. Custom layers/losses/metrics
24. Advanced CV/NLP
25. Production TensorFlow

Best Practice for Your Journey

For every set, follow this cycle:

Read concept
↓

Write small code
↓
Break the code intentionally
↓
Fix the error
↓
Explain it in your own words
↓
Build a mini-project
↓
Move to next set

Minimum Projects You Should Build

Build these gradually:

1. Manual linear regression with GradientTape
2. Keras linear regression
3. Binary classification model
4. Multiclass classification model
5. CNN image classifier
6. Transfer learning image classifier
7. Text sentiment classifier
8. Time-series forecasting model
9. Custom training loop model
10. SavedModel export project
11. TensorFlow Serving REST API project
12. TensorFlow Lite conversion project
13. Multi-GPU training project
14. Custom layer + custom loss project
15. End-to-end production-style TensorFlow project

Final Target

After completing this roadmap, you should be able to:

Build models
Train models
Debug models
Customize training
Create custom layers/losses/metrics
Use TensorBoard
Use tf.data pipelines
Handle image/text/time-series data
Use transfer learning
Save and reload models
Deploy with TensorFlow Serving
Convert to TensorFlow Lite
Train on GPU/multi-GPU
Optimize performance
Integrate TensorFlow with backend systems

For your background as a Spring Boot developer, the most valuable advanced endpoint is not only “training models,” but **training + exporting + serving + integrating with backend applications**. That is where TensorFlow becomes production-useful.